

UC - Lógica de Programação / Aula 03

O que vimos na última aula - 24/04/2026?

- Resumo do conceito de variável;
 - Vimos como usar o **Teste de Mesa** para entender como o algoritmo vai evoluindo e alterando as variáveis ao longo dos comandos;
 - Configuramos o VSCode para digitar nosso primeiro código JavaScript em uma página web;
 - Criar um *workspace* (local de trabalho → uma pasta para poder guardar nossos arquivos) direto no VSCode;
 - Instalamos duas extensões do VSCode: *Material Icon Themes* & *Live Server*;
 - Fizemos o exercício de nº 3 - Exibir um número, seu antecessor e sucessor.
-

Resumo

Visual Studio Code

O Visual Studio Code, ou VSCode, é um dos principais editores de códigos presentes hoje no mercado de programação, facilitando em muito a vida dos desenvolvedores. É um software da Microsoft que possui muitos recursos para criação e gerenciamento de projetos. Ele é compatível com os principais Sistemas Operacionais. [Consulte o site para informações de instalação.](#)

Com o software instalado e inicializado, ele possui uma interface tranquila de usar e muito intuitiva. Possui as principais características de qualquer software que você instala no seu computador, como: barra de menus, pesquisa rápida, janelas de navegação em formato de abas, etc.

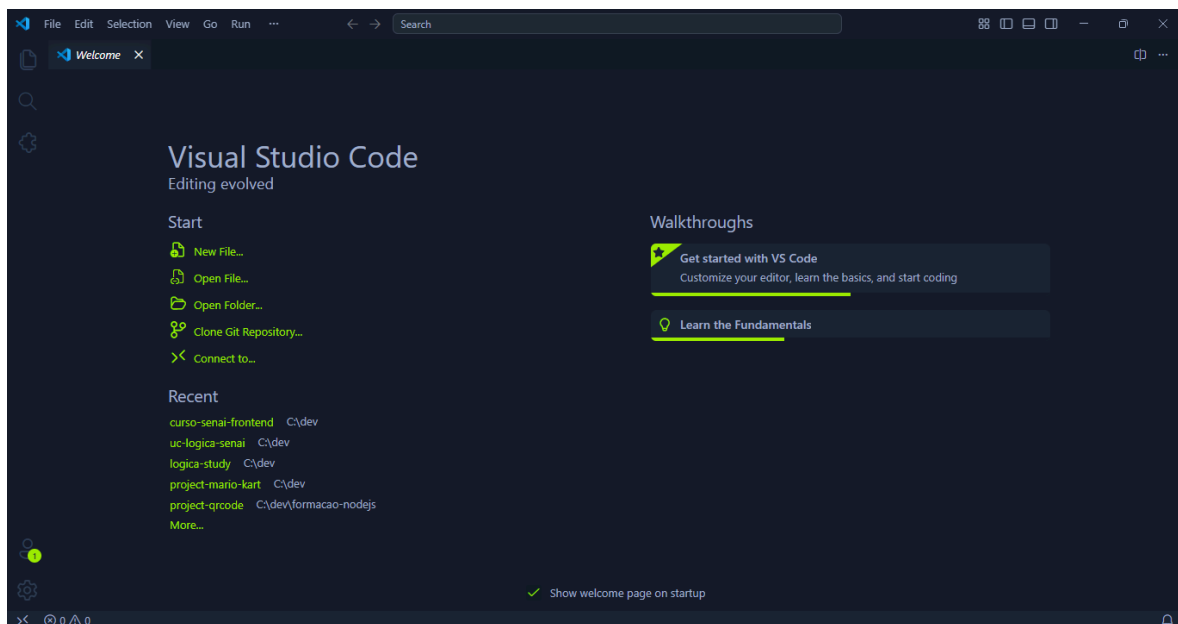


Imagem 01: Tela de Interface do Visual Studio Code.

- **Barra de Menu Superior**

A barra de menus é responsável por nos permitir acessar diversos recursos dentro do software.

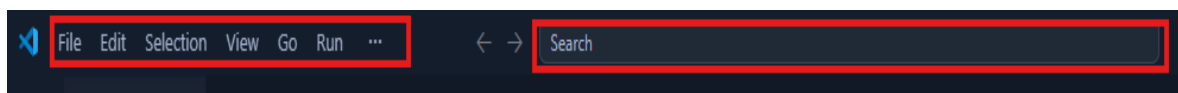


Imagem 02: Barra de Menu Superior.

Assim, podemos criar uma nova pasta, abrir pastas, salvar arquivos, configurar nossas preferências do VSCode e fazer buscas no campo de "Search".

- **Barra de Menu Lateral**

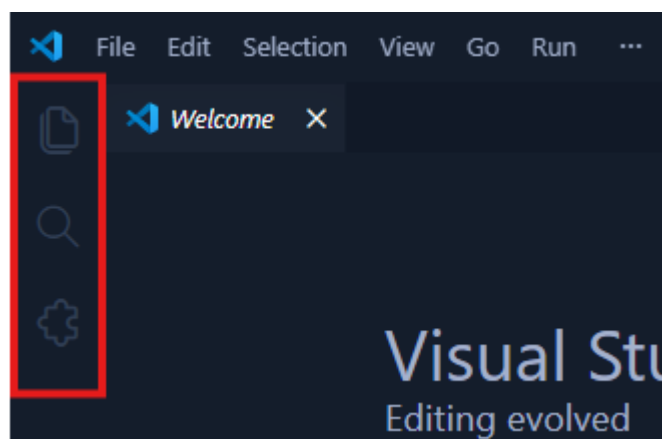


Imagem 03: Barra de Menu Lateral.

A **barra de menu lateral** tem algumas das principais ferramentas que vamos utilizar durante nosso progresso no curso. Confira os três que estão na imagem pela ordem de cima para baixo::

1. **Explorador de Arquivos (Explorer):** Atalho `Ctrl+Shift+E`.
 - a. Serve para gerenciar as pastas e arquivos do nosso projeto atual. Você pode clicar para abrir a visualização do Explorer e pode ocultar a visualização clicando novamente. Ou utilizando o atalho.
2. **Pesquisa (Search):** Atalho `Ctrl+Shift+F`.
 - a. Esta aba é uma poderosa ferramenta para localizar algum texto/comando de dentro do nosso projeto atual. É uma excelente ferramenta para poder localizar e substituir por outro texto rapidamente.
3. **Extensões (Extensions):** Atalho `Ctrl+Shift+X`.
 - a. Extensões no VS Code são plugins que adicionam novas funcionalidades, linguagens, ferramentas de depuração e temas ao editor. Elas permitem que você personalize o seu ambiente de trabalho de acordo com as suas necessidades específicas como programador.

Criando um *Workspace* (local de trabalho)

As páginas web que um desenvolvedor front-end trabalha, basicamente, utiliza as três principais tecnologias: **html** (**H**yper**T**ext **M**arkup **L**anguage - *Linguagem de Marcação de Hipertexto, em português*), **css** (**C**ascading **S**tyle **S**heet - *Folhas de Estilo em Cascata*) e **js** (**J**ava**S**cript - *Linguagem de Programação para trazer interatividades às páginas web*).

Saiba apenas que essa sopa de letrinhas significa apenas as definições destes arquivos, bem como sua utilização. Cada um deles possui suas regras de escrita. *Veremos mais sobre isto no decorrer do curso*. Apenas tenha em mente que são **formatos** de arquivos, assim como o PDF, PNG, JPEG. Portanto, segue exemplos de arquivos html, css e js:

- `pagina-inicial.html`
- `sobre.html`
- `styles.css`
- `server.js`

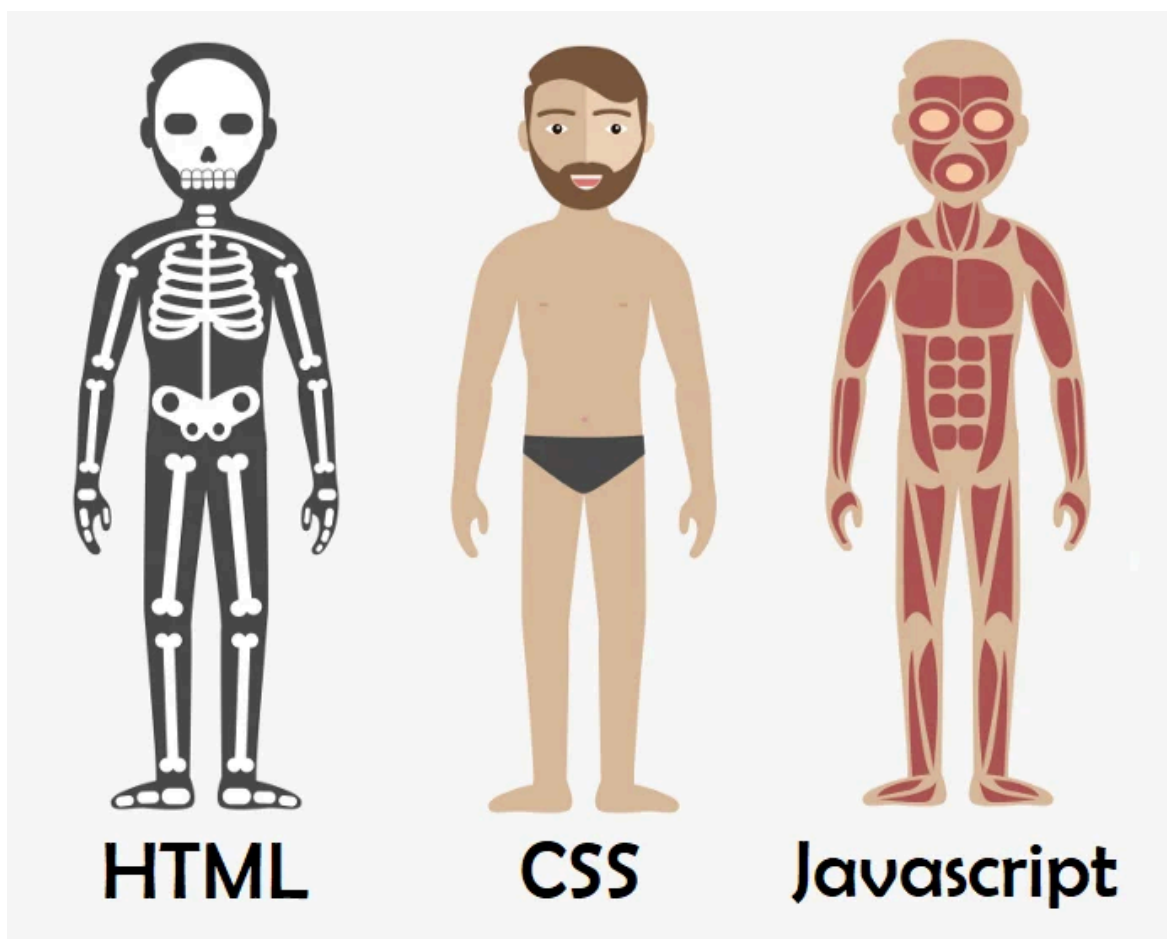


Imagem 04: Representação Visual das Principais Tecnologias de Páginas Web.

A imagem acima retrata exatamente como devemos visualizar uma página web sob a perspectiva de um desenvolvedor.

- **HTML:** É a estrutura da página. É nele que criamos os textos, inserimos imagens, links, vídeos, títulos, tabelas, listas, etc. **É a parte que dá o corpo inicial.**
- **CSS:** É responsável por dar estilos a nossa aplicação web. Como cores, tipos e tamanhos de fontes, animação, efeitos, etc. **É a parte que deixa nosso site bonitinho.**
- **JAVASCRIPT:** É a parte da dar corpo e vida à nossa página web. Que permite ao usuário interagir com a página. **É a parte que dá vida e deixa nossa página interativa, isto é, permite o usuário modificar, clicar e receber informações de respostas.**

E o workspace?

Workspace nada mais é que uma **pasta** onde vamos guardar nossa página web. É extremamente importante que organize suas pastas com nomes. Inclusive

existe uma forma de documentar e estruturar suas pastas. “No decorrer do curso, vamos aprendendo a fazer isto”. Por hora, vamos prosseguir:

Criar uma pasta ou Abrir uma já existente:

Você pode criar uma pasta em qualquer lugar do seu computador, ou pode fazer isto dentro do próprio editor de texto: VS Code. Aqui, sempre iremos utilizar o VS Code para gerenciar nossas pastas.

1. Acesse a **barra de menu superior** do seu VS Code e clique em **“File > Open Folder”**.

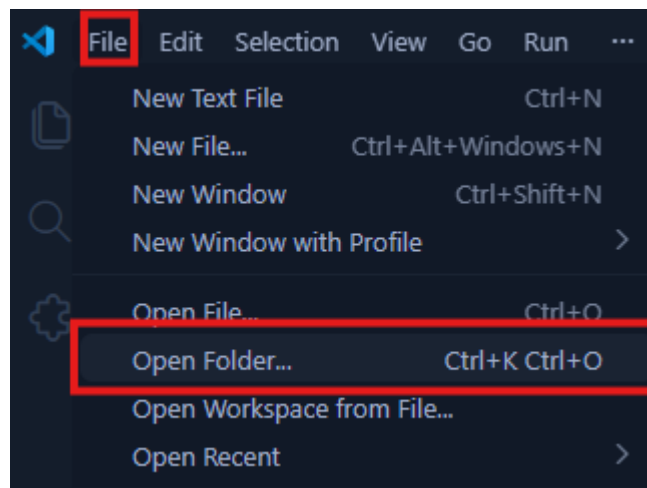


Imagem 05: Abrindo uma pasta no menu “File”.

2. Abrirá um explorador de arquivos do Windows. Basta selecionar a pasta que tenha seu projeto. E se for a primeira vez e você ainda não tem projeto: crie uma pasta e selecione ela em “Select Folder”.

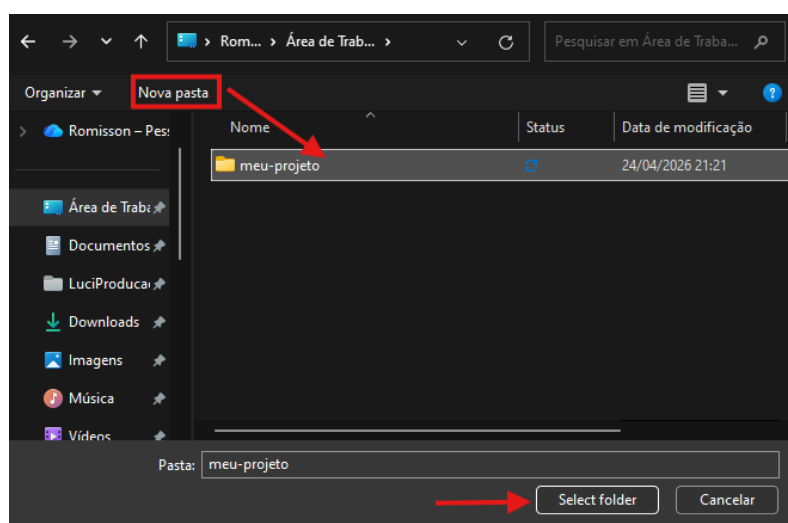


Imagem 06: Criando uma pasta e selecionando ela.

3. Agora que temos uma pasta e já abrimos ela: **temos nosso workspace criado - chamaremos de “pasta raiz”**.

Criando nossas pastas

Ao abrir a nossa pasta de trabalho, a **pasta raiz**, no VS Code e acessando o *Explorer*, podemos visualizar pastas e arquivos que tem dentro da pasta raiz. Por enquanto, não temos nada. Então vamos criar uma pasta chamada **aula-01**.

Dica para nomes:

Ao criar suas pastas :

- evitem nomes com espaço;
- evitem nomes com caracteres especiais - símbolos como parênteses etc;
- utilize, de preferência, letras minúsculas sem acentos e sem cedilha;
- para delimitar espaços, utilize um traço ou sublinhado (- ou _) entre as palavras: ex: minha-primeira-pagina, meu_site, etc.

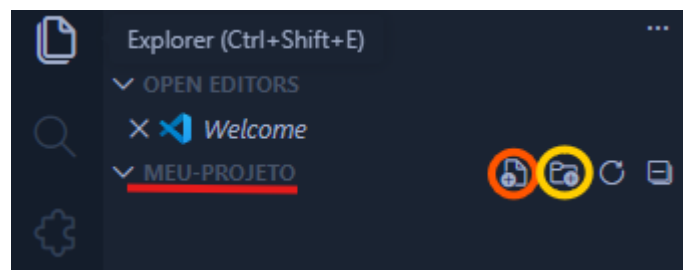


Imagem 07: Exibindo o Explorer.

Na imagem acima, temos:

- **meu-projeto:** sublinhado de vermelho, temos o nome e a pasta que acabamos de criar. Se você selecionar uma pasta para abrir, o nome dela aparecerá aqui.
- **new file** (novo arquivo): com círculo laranja, é o ícone rápido para criar um arquivo novo. Pode ser um arquivo do tipo txt, md, json, html, css, js, entre outros.
 - Para criar um arquivo, basta clicar neste item e digitar o nome + ponto (.) + extensão do arquivo. Vimos sobre isto acima, quando falamos da extensão. Ex: index.html, styles.css, index.js, readme.md, etc.

- **new folder** (nova pasta): com círculo amarelo, este ícone nos permite criar uma pasta rapidamente apenas clicando e digitando o nome. Para pastas, digite apenas o nome dela. Observe que tem uma seta ao lado do ícone da pasta.

Quando ela está apontada para a pasta, indica que a pasta está fechada. Agora **se ela estiver apontada para baixo**, fazemos isto clicando na pasta, significando **que estamos dentro dela**.

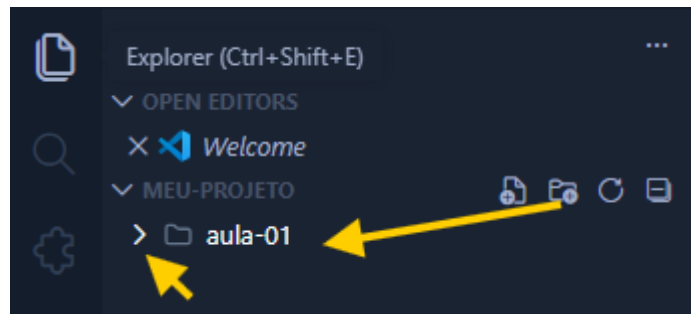


Imagem 08: Criando pasta "aula-01/".

"Como assim? Fiquei confuso 🤔! Calma, respira!"

O que eu quero dizer é: imagine que eu queira **criar** uma **pasta dentro** da **aula-01**. Se a seta **estiver apontada para frente**, e eu clicar no ícone que cria pasta, então o VS Code vai entender que você quer criar uma pasta na raiz, ou seja, "VS Code, crie uma pasta **ao lado** da **aula-01**". Elas seriam "pastas irmãs". Agora **se eu quiser criar uma pasta DENTRO da pasta "aula-01"**, aí, sim, deve clicar nela, verificar que a **seta está apontada para baixo** e clicar no ícone de "new folder" (nova pasta). Ou clique com o botão direito do mouse na pasta "aula-01" e clique em "new folder".

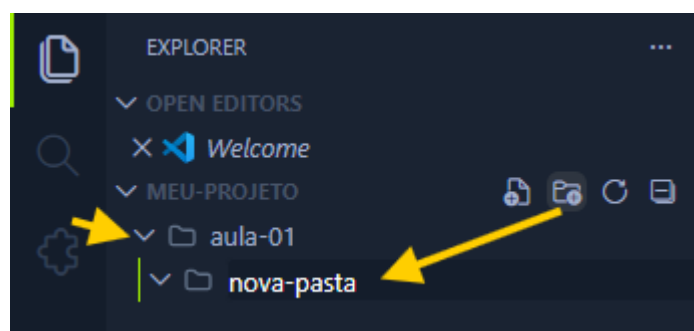


Imagem 10: Criando pasta dentro da pasta "aula-01/".

Criando nossos arquivos

Para criação de arquivos é tranquilo agora que temos pastas para organizar.

Para a resolução dos exercícios propostos durante nosso curso de lógica, vamos criar uma **estrutura básica** de página web para poder escrever nossos códigos JavaScript.

Dentro da pasta “aula-01/” (com a seta para baixo ao lado dela), clicamos no ícone de “new file” e digitamos o nome do arquivo. Vamos chamá-lo de **“primeira-pagina.html”** e aperte a tecla “Enter” do teclado.

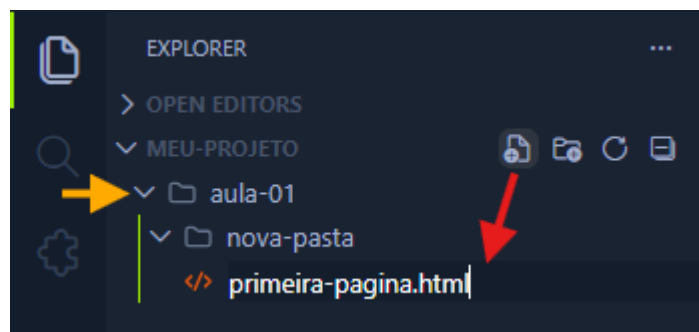


Imagem 11: Criando o arquivo “primeira-pagina.html” dentro da pasta “aula-01/”

O VS Code vai criar uma janela ao lado com o arquivo que acabamos de criar.

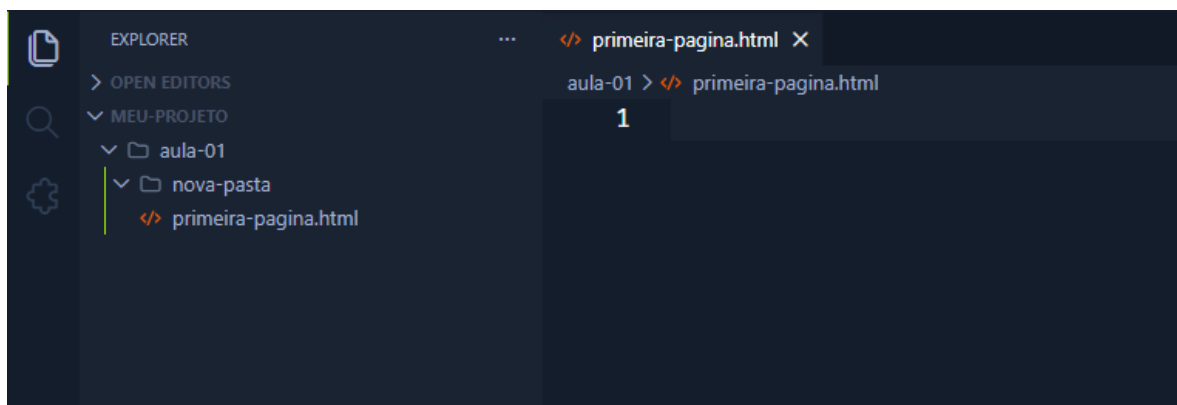


Imagem 12: Visualizando janela do arquivo “primeira-pagina.html”

Para fechar o visualizador do explorador de arquivos, clicamos no ícone da folhinha do Explorer na barra de menus lateral, ou digitando o atalho **Ctrl+Shift+E** - (Ver em [barra de menu lateral](#)). Assim, podemos visualizar apenas a janela que está aberta o arquivo que acabamos de criar.

Criando estrutura HTML básica e rápida

Conforme conversamos em sala de aula, no andamento do curso vamos ver mais a fundo o html para criação de páginas, porém, por hora, só precisamos de uma estrutura simples e rápida para rodar nossas aplicações JavaScript e poder realizar os exercícios propostos em sala de aula.

Emmet: O emmet é uma ferramenta nativa do VS Code responsável por facilitar nosso desenvolvimento. É um recurso fantástico com atalhos e “*auto complete*” e vamos fazer uso de alguns recursos. Devo **reforçar: sempre que possível digite seus códigos**, assim você cria memória cognitiva e acaba aprendendo muito mais. Como nosso foco não é HTML agora, vamos usar a ferramenta do *Emmet* para preencher para nós. **Nos códigos JS, escreva!**

Duas formas de iniciar nosso projeto HTML → **!** ou **html:5**

Com o arquivo que acabamos de criar, digite **!** ou **html**. Normalmente você verá um menu indicando que o Emmet quer completar para você. No caso do sinal de interrogação, basta pressionar “Enter” do teclado.

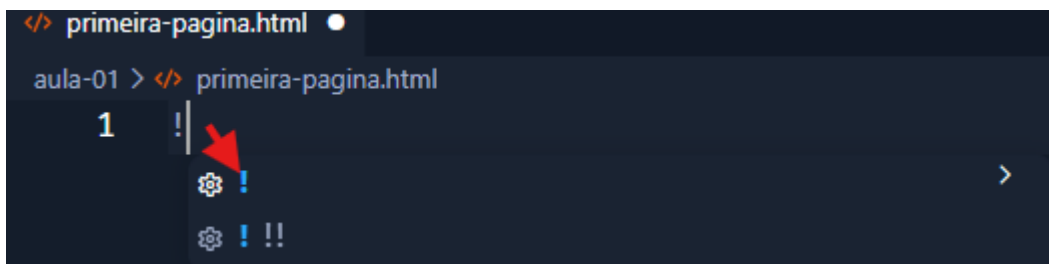


Imagem 13: Digitando ! no arquivo e visualizando as opções, clique em ! do menu que abriu.

Já o caso de digitar **html**, vai abrir várias opções, escolha **html:5**:

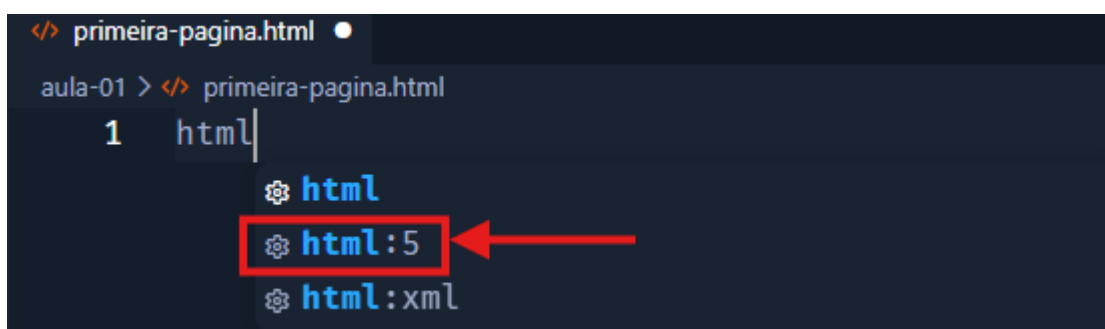


Imagem 14: Digitando html escolha a opção → html:5

Das duas formas, o Emmet vai preencher um padrão html básico pronto.

```

aula-01 > </> primeira-pagina.html > html
1  <!DOCTYPE html>
2  <html lang="en">
3  <head>
4      <meta charset="UTF-8">
5      <meta name="viewport" content="width=device-width, initial-scale=1.0">
6      <title>Document</title>
7  </head>
8  <body>
9
10 </body>
11 </html>

```

Imagem 15: Estrutura básica de um arquivo HTML

Não se preocupem com todos estes códigos. Por ora, vamos apenas fazer pequenas alterações:

1. **Idioma:** para definir que nossa página está em português do Brasil, onde está escrito `<html lang='en'>`, vamos substituir o valor `'en'` por `'pt-BR'`. Vamos substituir apenas o texto e vamos deixar entre aspas.
2. **Título da Página:** para definir o título da página, aquela que aparece lá na aba do navegador, onde está `<title>Document</title>` vamos remover o texto `Document` e escrever o nome da nossa página `Minha Página`. **Repare que Document está sem aspas, então vamos digitar o nosso texto sem aspas também.**
3. **Um texto para aparecer na página:** abaixo de `<body>`, vamos escrever um texto direto `Olá, mundo!` - sem aspas também.

```

primeira-pagina.html X
aula-01 > </> primeira-pagina.html > html > body
1  <!doctype html>
2  <html lang="pt-BR">
3      <head>
4          <meta charset="UTF-8" />
5          <meta name="viewport" content="width=device-width, initial-scale=1.0" />
6          <title>Minha Página</title>
7      </head>
8      <body>
9          Olá, mundo!
10     </body>
11 </html>

```

Imagem 16: Definindo idioma, título e conteúdo da nossa página web

Criando local da página para escrever nossos códigos JavaScript

Perfeito, agora temos uma página já configurada para abrir no navegador e começar a praticar JavaScript.

Mas onde eu escrevo o código JavaScript agora?

Para isto, precisamos demarcar um local para inserir nossos comandos, tudo que já aprendemos até agora: criar variável para guardar dados, exibir informações no console do navegador ou exibir um alerta (*alert*) para usuário etc.

Tag `<script>`

A tag `<script>` é como vamos indicar para o navegador que pegar nosso arquivo recém criado (*primeira-pagina.html*) que o que **estiver entre `<script>` e `</script>` é um código JavaScript** e portanto deve ser executado **linha a linha** cada comando digitado nele.

Observe como fazemos isto:



```
1 <script>
2     console.log("Olá, desenvolvedor");
3 </script>
```

Imagem 17: Escrevendo código JavaScript dentro da tag `<script>`

- `<script>` → diz: “aqui começa um bloco de instruções JavaScript”;
- *conteúdo depois de `<script>`* → vai ser inserido nosso algoritmo na sintaxe do JavaScript para o motor de interpretação do navegador possa rodar a linguagem lá.
- `</script>` → é uma **tag de fechamento**. Indica para o navegador: “aqui se encerra um bloco de instruções JavaScript”.

? E onde inserimos a tag <script> ?

- Dentro da tag `<body>` antes de fechar (`</body>`). Observe:



```
1 <!doctype html>
2 <html lang="pt-BR">
3 <head>
4   <meta charset="UTF-8" />
5   <meta name="viewport" content="width=device-width, initial-scale=1.0" />
6   <title>Minha Página</title>
7 </head>
8 <body>
9   Olá, mundo!
10
11 <script>
12   console.log("Olá, desenvolvedor");
13 </script>
14 </body>
15 </html>
```

Imagem 18: Inserindo a tag script e adicionando comando JavaScript

Confira o bloco de código por texto:

meu-projeto/aula-01/primeira-pagina.html

```
<!doctype html>
<html lang="pt-BR">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width,
initial-scale=1.0" />
    <title>Minha Página</title>
  </head>
  <body>
    Olá, mundo!

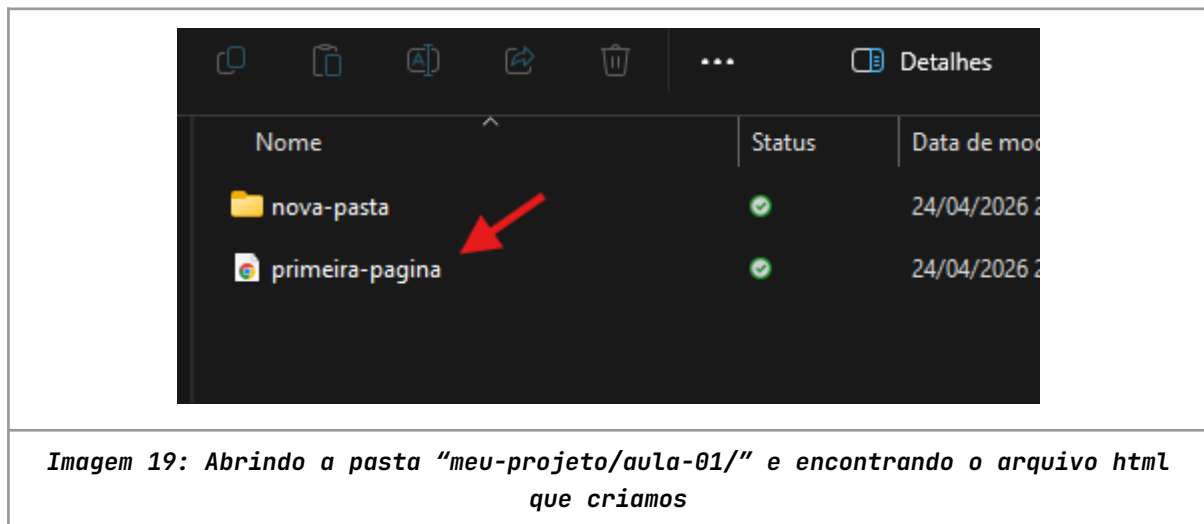
    <script>
      console.log("Olá, desenvolvedor");
    </script>
  </body>
</html>
```

Rodando nossa página no navegador

Basicamente, temos várias formas de fazer isto. Vamos ver as duas principais:

1. Abrindo o arquivo “.html”:

Localize o arquivo que você acabou de criar e digitar (“*primeira-pagina.html*”) lá na pasta raiz direto no seu computador. No nosso exemplo, está na pasta “*meu-projeto*”. E está dentro da pasta “*aula-01*”.



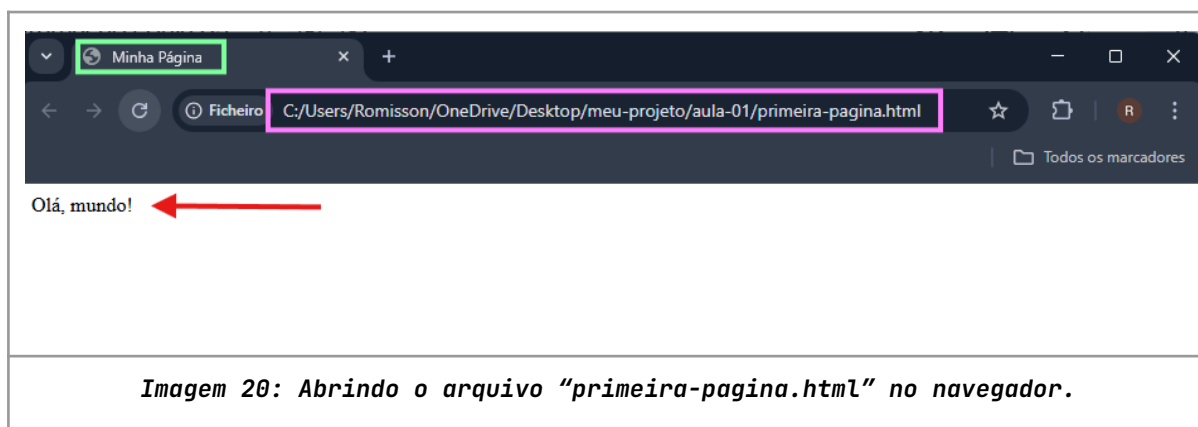
👁 Observe:

Nós estamos representando uma pasta com uma barra “/” no final de seu nome. Esta é uma forma de representar “diretórios”: pastas.

Assim, quando escrevemos, por exemplo, “*meu-projeto/aula-01/*” estamos indicando um caminho de arquivo: **dentro da pasta “meu-projeto” tem uma pasta chamada “aula-01”.**

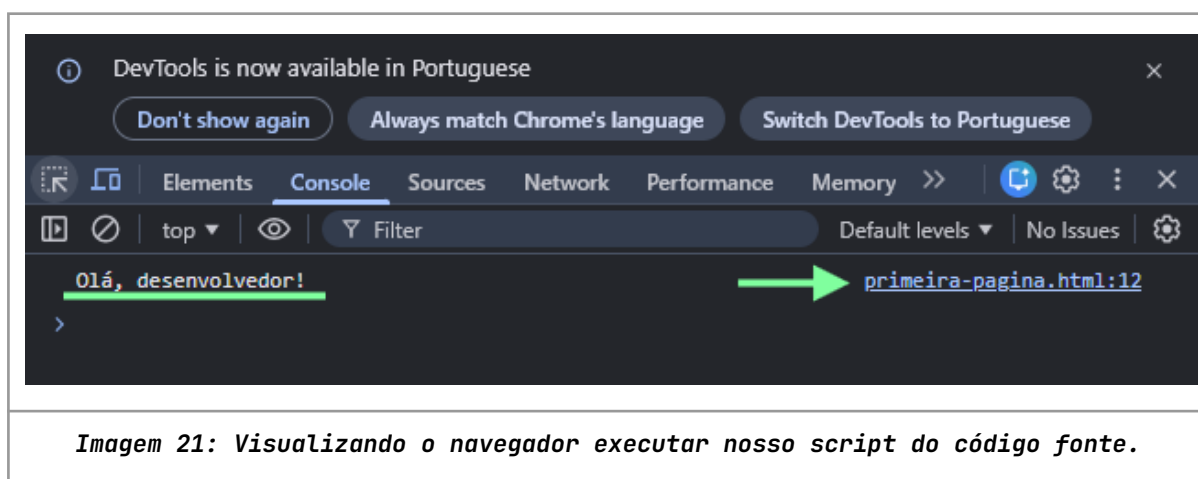
E para representar um arquivo, indicamos o nome do arquivo + a extensão. Portanto, o “caminho” deste arquivo final é: **“meu-projeto/aula-01/primeira-pagina.html”.**

Ao clicar duas vezes no arquivo que acabamos de criar, vai abrir, no navegador de sua preferência (aquele que você definiu como padrão) a abertura de uma página web.



- "Minha Página": é o que escrevemos entre as tags de `title` → `<title>Minha Página</title>`;
- `"file:///C:/Users/Romisson/OneDrive/Desktop/meu-projeto/aula-01/primeira-pagina.html"`: é o caminho do nosso arquivo.
- `"Olá, mundo!"`: é o conteúdo que inserimos dentro da tag `<body>`.

E o console que executamos na tag script? Se acessarmos a ferramentas do desenvolvedor (F12), ou Inspecionar, na nossa primeira página, na aba *console*, temos a execução do código que JavaScript da nossa página web



2. Executando com extensão Live Server:

Vimos também que podemos rodar um "servidor" na nossa máquina para enviar os arquivos do nosso projeto para o navegador. E para fazer isso, instalamos o Live Server. Ele vai simular como se uma máquina tivesse a cópia do nosso arquivo e vai enviar ao navegador quando o mesmo fizer uma solicitação pedindo para ver a página. E para fazer isto localmente, ele utiliza sua própria máquina acessando uma das "portas" de comunicação, normalmente a **porta 5500**.

Não se preocupem! O próprio *Liver Server* faz isto para a gente com apenas um clique. Mas antes, precisamos instalar a extensão.

Instalando a extensão **Live Server**

Para instalar a extensão, acesse a **barra de menu lateral** e clique em “Extensions”, ou digite o atalho: “**Ctrl+Shift+X**”:

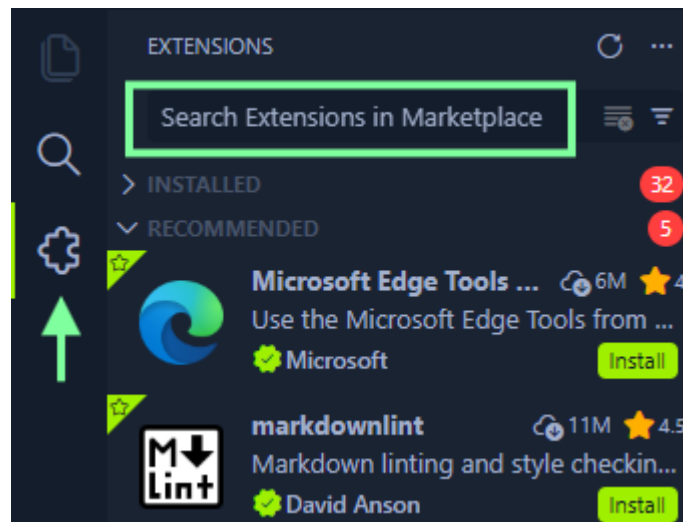


Imagem 22: Acessando o menu “Extensions” no VS Code

- Onde está escrito “Search Extensions in Marketplace”: digite “Live Server”. E instale a extensão.

Depois de instalado (algumas vezes você terá que fechar e abrir o VS Code novamente), vai aparecer um botão na parte inferior da interface do VS Code escrito “**Go Live**”. Clique neste botão, com a página html aberta.

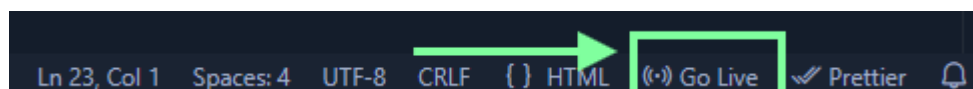


Imagem 23: Acessando o menu “Extensions” no VS Code

Normalmente, o Live Serve já abre uma página no seu navegador. Se ele não fizer, observe que o “Go Live” vai ser substituído pela informação que qual porta a extensão estará servindo nossa aplicação. Na maioria das vezes ele aparece: “Port: 5500”. Ou seja, ele estará “servindo” uma cópia para o navegador que pedir neste endereço.

Portanto, enquanto o servidor estiver ativo, basta digitar na **url** de qualquer **navegador do seu computador**, o seguinte:

```
localhost://5500/<caminho-do-seu-projeto>/.
```

No nosso caso o caminho é: “aula-01/primeira-pagina.html”. Confira:

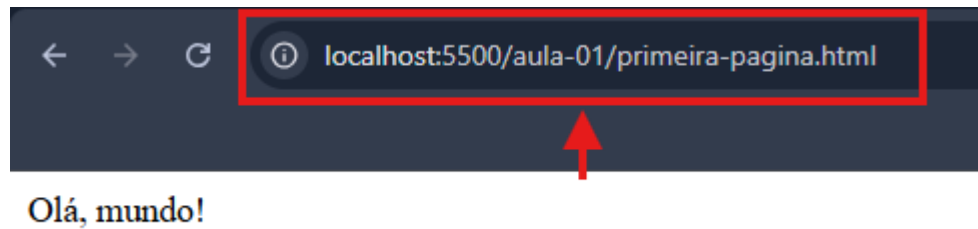


Imagem 24: Digitando o “caminho” do servidor para receber uma cópia da nossa aplicação.

👁 Dica:

Se você não souber o caminho da sua aplicação e o Live Server não abrir a página em um navegador, não tem problema!

Basta digitar apenas `localhost:5500/` no seu navegador **com o Live Server** ativo e rodando. Você vai ver uma página com ícones de pastas e arquivos, esta página é uma representação da sua pasta raiz. Basta navegar até encontrar o arquivo que você quer abrir.

Localhost significa “sua máquina local”, em outras palavras: **seu computador**.

Resolução do Exercício #3

Com todo conhecimento que conversamos hoje, podemos resolver o exercício proposto e ver a saída no console do navegador:

Enunciado: Crie um programa que guarda um número e exibe o número, seu antecessor e seu sucessor.

meu-projeto/aula-01/ex03.html

```
<!doctype html>
<html lang="pt-BR">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0"
  />
    <title>Exercício #3</title>
  </head>
  <body>
    Enunciado: Crie um programa que guarda um número e exibe o número, seu
    antecessor e seu sucessor.

    <script>
      let num = 98;
      let antecessor = num - 1;
      let sucessor = num + 1;

      console.log(num);
      console.log(antecessor);
      console.log(sucessor);
    </script>
  </body>
</html>
```

Agora basta abrir o arquivo .html ou rodar o Live Server para verificar a saída no *console* do navegador.

Bons estudos!!!